How LLM Parameters Are Stored in Memory

A gentle introduction to float32, float16, bfloat16, and 4-bit quantization

Abstract

Large language models (LLMs) contain billions of parameters, yet every one of these parameters must ultimately be stored as bits in memory. This note explains how common numerical formats—*float32*, *float16*, *bfloat16*, and simple *4-bit quantization*—represent real-valued weights. We walk through an explicit float32 example at the bit level, compare float16 and bfloat16, and construct a small numerical example of 4-bit quantization, then estimate memory usage for a 1.7B-parameter model under each scheme.

1 Floating-point numbers: general idea

In deep learning code we think of parameters as real numbers $w \in \mathbb{R}$, such as $w = 0.786473$. However, in RAM or VRAM each parameter is stored as a *fixed-length pattern of bits* according to some numeric format. For floating-point types (float32, float16, bfloat16), that format is a variant of the IEEE-754 standard.

Conceptually, a floating-point value is represented as

$$v = (-1)^{\text{sign}} \cdot (1.\text{fraction})_2 \cdot 2^{\text{exponent}-\text{bias}},$$

where $(1.\text{fraction})_2$ denotes a binary fraction and the *bias* is a constant (e.g. 127 for float32) used to encode signed exponents as unsigned bit patterns.

The important points are:

- The number of bits allocated to the exponent controls the *dynamic range*.
- The number of bits allocated to the fraction (mantissa) controls the *precision*.
- Different formats trade range against precision and memory footprint.

2 Float32 by example

The most common reference format is *float32* (single precision):

- 1 sign bit,
- 8 exponent bits (bias = 127),
- 23 fraction bits.

This gives 32 bits (4 bytes) per value.

We now work a concrete example all the way from a decimal value to the raw bit pattern and back.

Example: representing 0.15625 in float32

Consider the decimal number $v = 0.15625$. We first write it as a binary fraction. Note that

$$0.15625 = \frac{1}{8} + \frac{1}{32} = 0.125 + 0.03125,$$

and using powers of two,

$$\tfrac{1}{8} = 2^{-3} = 0.001_2, \qquad \tfrac{1}{32} = 2^{-5} = 0.00001_2.$$

Therefore

$$0.15625_{10} = 0.00101_2.$$

We normalize to the form $1.\text{fraction} \times 2^e$:

$$0.00101_2 = 1.01_2 \times 2^{-3}.$$

So

- sign $= 0$ (positive),
- exponent $e = -3$,
- normalized significand $= 1.01_2$.

For float32 the stored exponent field is

$$E = e + \text{bias} = -3 + 127 = 124.$$

In binary, $124 = 0\,1111\,100_2$, so the exponent bits are `01111100`.

The fraction field stores the bits *after* the leading 1. Our normalized significand is 1.01_2, so the fraction is 0.01_2 followed by zeros:

$$\text{fraction bits} = 01000000000000000000000.$$

Collecting all three fields (sign, exponent, fraction), we obtain the 32-bit pattern

$$\underbrace{0}_{\text{sign}}\ \underbrace{01111100}_{\text{exponent}}\ \underbrace{01000000000000000000000}_{\text{fraction}}.$$

This is exactly what lives in memory for the float32 value 0.15625.

Decoding the bits back to decimal

Starting from the bit fields:

- sign bit $= 0 \Rightarrow (+1)$,
- exponent bits `01111100` $= 124 \Rightarrow e = 124 - 127 = -3$,
- fraction bits $0100\ldots0$ correspond to $0.01_2 = 0.25$.

The significand is $1 + 0.25 = 1.25$. Thus

$$v = (+1) \cdot 1.25 \cdot 2^{-3} = 1.25 \times 0.125 = 0.15625,$$

which matches the original decimal value.

This example shows how a single float32 weight is just a particular arrangement of 32 bits. For an LLM with N parameters, the weights alone require roughly $4N$ bytes of memory in this format.

3 Float16 and bfloat16

To reduce memory and bandwidth requirements, modern LLMs often use 16-bit floating-point formats for computation. Two important ones are IEEE float16 and bfloat16.

IEEE float16

The IEEE float16 layout is:

- 1 sign bit,
- 5 exponent bits (bias = 15),
- 10 fraction bits.

Compared to float32 this yields:

- smaller exponent range (fewer extreme values representable),
- less precision (about $2^{10} \approx 10^3$ discrete steps in the significand),
- *half* the memory per parameter (2 bytes instead of 4).

For many neural networks, this is sufficient during training and inference, though underflow/overflow can be more frequent than with float32.

bfloat16

The bfloat16 format rearranges bits differently:

- 1 sign bit,
- 8 exponent bits (same as float32, bias = 127),
- 7 fraction bits.

This keeps *the same exponent range* as float32, but with reduced precision. The trade-off is:

- similar robustness to very large/small values as float32,
- lower precision in the mantissa than float32 or even float16,
- still only 2 bytes per parameter.

Because LLMs often benefit more from a wide dynamic range than from extremely fine precision, bfloat16 has become a popular compute format: activations and many intermediate results are stored in bfloat16, while weights may be in bfloat16 or in even more compact quantized formats.

4 A small 4-bit quantization example

Floating-point formats store each parameter independently, but we can also compress groups of parameters by quantizing them to low-bit integers and storing a shared scale. Here we illustrate a simple 4-bit symmetric quantization scheme.

Setup

Suppose a layer has five scalar weights

$$w = [-1.00,\ -0.52,\ 0.10,\ 0.73,\ 1.20].$$

We will approximate each w_i with a 4-bit integer code $q_i \in \{-7, \ldots, 7\}$ and a shared positive scale s. The reconstructed (dequantized) weight is

$$\hat{w}_i = s \cdot q_i.$$

Choosing the scale

We map the maximum absolute weight to the maximum code magnitude 7:

$$\max_i |w_i| = 1.20, \qquad s = \frac{\max_i |w_i|}{7} = \frac{1.20}{7} \approx 0.1714.$$

Thus one integer step corresponds to approximately 0.1714 in real units.

Quantization: float to 4-bit integer

We define

$$q_i = \text{clip}_{[-7,7]}\Big(\text{round}\Big(\frac{w_i}{s}\Big)\Big).$$

Computing each q_i:

$$\begin{aligned}
w_1 = -1.00 &\Rightarrow -1.00/0.1714 \approx -5.83 &&\Rightarrow q_1 = -6,\\
w_2 = -0.52 &\Rightarrow -0.52/0.1714 \approx -3.04 &&\Rightarrow q_2 = -3,\\
w_3 = 0.10 &\Rightarrow 0.10/0.1714 \approx 0.58 &&\Rightarrow q_3 = 1,\\
w_4 = 0.73 &\Rightarrow 0.73/0.1714 \approx 4.26 &&\Rightarrow q_4 = 4,\\
w_5 = 1.20 &\Rightarrow 1.20/0.1714 \approx 7.00 &&\Rightarrow q_5 = 7.
\end{aligned}$$

Each q_i now fits into 4 bits.

Dequantization: 4-bit integer back to float

We reconstruct approximate weights via $\hat{w}_i = s \cdot q_i$:

$$\begin{aligned}
\hat{w}_1 &= 0.1714 \times (-6) \approx -1.028,\\
\hat{w}_2 &= 0.1714 \times (-3) \approx -0.514,\\
\hat{w}_3 &= 0.1714 \times 1 \approx 0.171,\\
\hat{w}_4 &= 0.1714 \times 4 \approx 0.686,\\
\hat{w}_5 &= 0.1714 \times 7 \approx 1.200.
\end{aligned}$$

The original and reconstructed values are summarized in Table 1.

Index	Original w_i	Code q_i	Dequantized $\hat{w}_i$
1	-1.00	-6	-1.028
2	-0.52	-3	-0.514
3	0.10	1	0.171
4	0.73	4	0.686
5	1.20	7	1.200

Table 1: Toy 4-bit quantization example with shared scale $s \approx 0.1714$.

In a real LLM, this process happens for large weight tensors, often with a separate scale per channel or per small block of weights. At inference time, the 4-bit codes and scales are stored in memory; when a layer is evaluated, the codes are dequantized on the fly to a floating-point type (e.g. bfloat16) in fast registers, used in matrix multiplications, and then discarded.

5 Memory usage for a 1.7B-parameter model

Let N denote the number of trainable parameters. SmolLM2-1.7B has roughly $N = 1.7 \times 10^9$ parameters. Ignoring activations and other buffers, the weight storage cost under different formats is approximately:

Per-parameter cost

- float32: 32 bits = 4 bytes,
- bfloat16: 16 bits = 2 bytes,
- 4-bit quantization: 4 bits = 0.5 bytes for the code, plus overhead for scales and metadata.

With one float32 scale per 128 weights, 4-bit quantization uses roughly

$$0.5 + \frac{4}{128} \approx 0.53 \text{ bytes per parameter},$$

so we round to about 0.55 bytes/parameter to account for additional metadata.

Total weight memory

For $N = 1.7 \times 10^9$ parameters we obtain the estimates in Table 2.

Format	Bytes per parameter	Total for $N = 1.7 \times 10^9$
float32	4	$\approx 6.8 \times 10^9$ bytes $\approx$ 6.8 GB
bfloat16	2	$\approx 3.4 \times 10^9$ bytes $\approx$ 3.4 GB
4-bit quant	≈ 0.55	$\approx 0.94 \times 10^9$ bytes $\approx$ 0.9–1.0 GB

Table 2: Approximate weight memory for a 1.7B-parameter model under different formats.

These figures explain why 4-bit quantization is so attractive for inference: a 1.7B-parameter model's weights can fit comfortably within about one gigabyte of VRAM, leaving room for activations, key–value caches, and other runtime overhead, even on relatively modest GPUs.

6 Summary

From the perspective of memory, LLM parameters are not mysterious: each weight is just a structured pattern of bits. Float32 devotes many bits to a wide dynamic range and fine precision; float16 and bfloat16 halve the memory footprint while trading some precision or exponent range; 4-bit quantization goes further by representing groups of weights as low-bit integer codes plus shared scales.

These numerical tricks are what make it feasible to store and run billion-parameter models on commodity hardware. Understanding how the formats work clarifies how frameworks like PyTorch and libraries like Unsloth can load, quantize, and execute LLMs efficiently.